

JARVIS-1: Open-World Multi-Task Agents With Memory-Augmented Multimodal Language Models

Zihao Wang[✉], Shaofei Cai[✉], Anji Liu, Yonggang Jin, Jinbing Hou, Bowei Zhang[✉], Haowei Lin, Zhaofeng He[✉], Zilong Zheng, *Member, IEEE*, Yaodong Yang[✉], Xiaojian Ma[✉], and Yitao Liang[✉], *Member, IEEE*

Abstract—Achieving human-like planning and control with multimodal observations in an open world is a key milestone for more functional generalist agents. Existing approaches can handle certain long-horizon tasks in an open world. However, they still struggle when the number of open-world tasks could potentially be infinite and lack the capability to progressively enhance task completion as game time progresses. We introduce JARVIS-1, an open-world agent that can perceive multimodal input (visual observations and human instructions), generate sophisticated plans, and perform embodied control, all within the popular yet challenging open-world Minecraft universe. Specifically, we develop JARVIS-1 on top of pre-trained multimodal language models, which map visual observations and textual instructions to plans. The plans will be ultimately dispatched to the goal-conditioned controllers. We outfit JARVIS-1 with a multimodal memory, which facilitates planning using both pre-trained knowledge and its actual game survival experiences. JARVIS-1 is the existing most general agent in Minecraft, capable of completing over 200 different tasks using control and observation space similar to humans. These tasks range from short-horizon tasks, e.g., “chopping trees” to long-horizon ones, e.g., “obtaining a diamond pickaxe”. JARVIS-1 performs exceptionally well in short-horizon tasks, achieving nearly perfect performance. In the classic long-term task of ObtainDiamondPickaxe, JARVIS-1 surpasses the reliability of current state-of-the-art agents by 5 times and can successfully complete longer-horizon and more challenging tasks. Furthermore, we show that JARVIS-1 is able to *self-improve* following a life-long learning

paradigm thanks to multimodal memory, sparking a more general intelligence and improved autonomy.

Index Terms—Minecraft, multimodal language model, open-world agents.

I. INTRODUCTION

CREATING sophisticated agents that can accomplish myriad of tasks in complex domains remains a pivotal milestone towards generally capable artificial intelligence [3], [7], [9], [43], [59]. Recent advancements have shown a trend towards employing a hierarchical goal execution architecture [22], [23], [50], and leveraging large language models (LLMs) as the high-level planner to generate action plans that will be ultimately executed by low-level instruction-following controllers. Albeit the fruitful progress they have yielded in many robotics [23] and even open-world environments like Minecraft [16], [20], today’s agents built with these approaches are still struggling with three major issues: 1) perceive the world from multimodal sensory observations, such as images, videos in addition to natural language instructions and feedback for planning; This is mostly due to the inability of LLM-based planners on processing multimodal data [22], [54]; 2) perform consistent and accurate long-term planning. This requires multi-round, knowledge, and reasoning-intensive dialogues, which remain great challenges to LLMs [23]; 3) learn and evolve in a life-long fashion. This calls out the need for agents to propose their own tasks and self-improve. Addressing these issues will unleash the full planning potential of LLM-based agents, and expedite the development of more generalist agents.

In this work, we introduce JARVIS-1, a brand new agent that can robustly produce plans for long-horizon tasks from multimodal user and environment inputs, and translate them into motor control in Minecraft, a popular yet challenging open-world testbed for generalist agents. To be specific, we chain a multimodal foundation model MineCLIP [16] and an LLM [9] together, the resulting multimodal language model (MLM) allows our agent to better understand the tasks, situations, and environmental feedback. To further enhance the correctness and consistency of planning, especially on long-horizon tasks, we propose to augment the agent with a *multimodal memory*, which stores both the scenarios and actual plans of the successful planning experiences in the past. By retrieving the relevant memory entries, the planning skill of our MLM-based agent can be strengthened from the agent’s own interactions with the

Received 20 November 2023; revised 29 August 2024; accepted 24 November 2024. Date of publication 5 December 2024; date of current version 5 February 2025. This work was supported in part by the National Key R&D Program of China under Grant 2022ZD0160301, in part by the NSFC under Grant 62376031, in part by CCF-Tencent Rhino-Bird Open Research Fund, in part by the NSF under Grant IIS-1943641, Grant IIS-1956441 and Grant CCF-1837129, in part by SRA from Meta, in part by a research gift from Amazon Alexa AI, and in part by a gift from Relational AI. Recommended for acceptance by W. Zhang. (Corresponding authors: Xiaojian Ma; Yitao Liang.)

Zihao Wang, Shaofei Cai, Bowei Zhang, Haowei Lin, Yaodong Yang, and Yitao Liang are with the Institute for Artificial Intelligence, Peking University, Beijing 100871, China (e-mail: zhwang@stu.pku.edu.cn; caishaofei@stu.pku.edu.cn; yitao@pku.edu.cn; yaodong.yang@outlook.com; yitao@pku.edu.cn).

Anji Liu is with the University of California, Los Angeles, CA 90095 USA (e-mail: liuanji@cs.ucla.edu).

Yonggang Jin, Jinbing Hou, and Zhaofeng He are with the School of Computer Science, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: daze@bupt.edu.cn; hjinbing@bupt.edu.cn; zhaofenghe@bupt.edu.cn).

Zilong Zheng and Xiaojian Ma are with the Beijing Institute for General Artificial Intelligence (BIGAI), Beijing 100876, China (e-mail: z.zheng@ucla.edu; xiaojian.ma@ucla.edu).

The project can be found in <https://craftjarvis-jarvis1.github.io/>.

This article has supplementary downloadable material available at <https://doi.org/10.1109/TPAMI.2024.3511593>, provided by the authors.

Digital Object Identifier 10.1109/TPAMI.2024.3511593

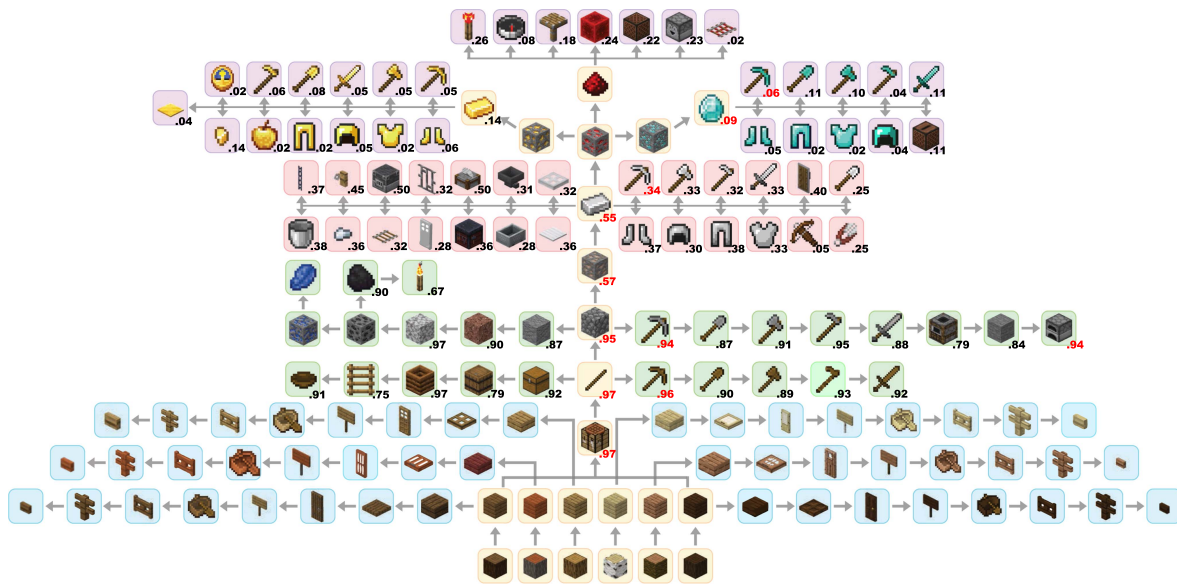


Fig. 1. How does JARVIS-1 unlock the technology tree of the Minecraft universe. JARVIS-1 can consistently obtain high-level items on the main tech-tree of the overworld in Minecraft, such as diamond, redstone, and golden items, which require collecting over 10 different intermediate items. JARVIS-1 not only outperforms the previous state-of-the-art VPT [4] (6% vs. 2.5% reliability within 20 minutes) on diamond pickaxe but also can craft almost all diamond items in overworld including diamond chestplate.

environment in an in-context manner. Finally, JARVIS-1 is able to evolve throughout the gameplay by proposing tasks on its own (i.e. self-instruct) as a means of exploration and saving the obtained experiences in the multimodal memory, therefore facilitating better reasoning and planning. This self-improving ability sparks its potential for a higher level of autonomy.

Our main evaluations are conducted in Minecraft, with more than 200 tasks selected from the Minecraft Universe Benchmark [31], with no demonstration provided. The tasks cover a broad spectrum from the early game (e.g. ObtainCraftingTable) to intermediate and even challenging long-horizon tasks (e.g. ObtainDiamondPickaxe). A glimpse of what JARVIS-1 is able to achieve can be found in Fig. 1. JARVIS-1 exhibits strong performances on these tasks, representing an up to $5\times$ increase to the previous records. Our ablative analysis then offers a detailed account of how JARVIS-1 approaches this significant progress and becomes the first agent that can robustly obtain the diamond pickaxe with up to 12.5% success rate. What is even more surprising is that, without the need for additional training, JARVIS-1 demonstrates a continuous increase in performance as game time increases in long-horizon tasks. Moreover, JARVIS-1 has demonstrated its potential of *self-improve* in an exploratory life-long learning experiment, where it needs to propose tasks to progressively explore the world, collect experiences, and sharpen its planning skill using these experiences stored in the multimodal memory.

In summary, JARVIS-1 pilots the effort towards a human-like multi-task and autonomous agent in an open-world, embodied environment like Minecraft. We would like to share the key takeaways of what we have learned during its development as follows:

- *From LLMs to MLMs*: The capability of perceiving multimodal sensory input is critical to planning in a dynamic

and open-world world. JARVIS-1 enables this by chaining a multimodal foundation model together with an LLM. Compared to LLM “blindly” produces plans, MLM is able to natively understand the current situation and plan accordingly. Further, rich environmental feedback can be obtained through multimodal perception, therefore helping the *self-check* and *self-explain* of the planner spot and fix possible bugs in the plans, enabling stronger interactive planning.

- *Multimodal memory*: Early research has suggested the crucial role that memory mechanisms can serve in the functioning of generalist agents. By outfitting JARVIS-1 with a multimodal memory, we effectively allow it to plan with both pretrained knowledge and its actual experiences in the world, therefore bringing significant improvement to planning correctness and consistency. Compared to canonical RL or planning agents with exploration, no additional model update is needed as the MLM in JARVIS-1 makes it possible to leverage these experiences in an in-context manner.
- *Self-instruct and self-improve*: A sign of generalist agents is the capacity to proactively acquire new experiences and continuously improve themselves. We have demonstrated how JARVIS-1 effectively traverses the environment by executing tasks autonomously generated through its *self-instruct* mechanism. With multimodal memory teaming up with experiences from the explorations, we have observed consistent improvement, especially in accomplishing more complicated tasks. Ultimately, this aspect of autonomous learning in JARVIS-1 signifies an evolutionary step towards generalist agents that can learn, adapt, and improve over time with minimal external intervention.

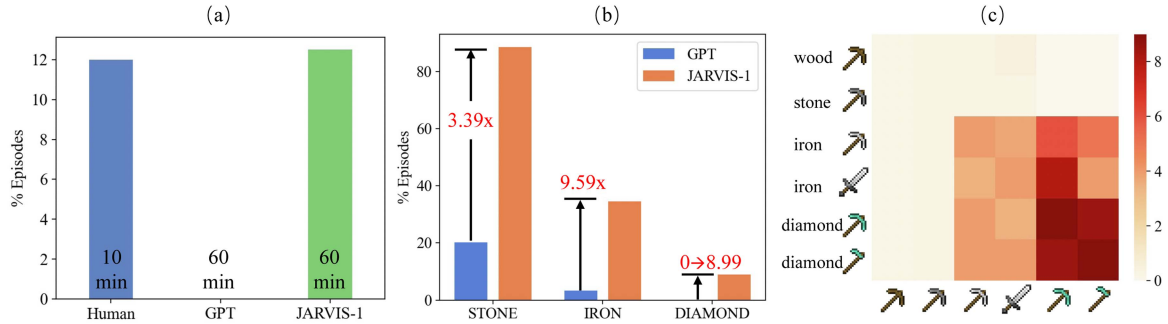


Fig. 2. Challenges in open-world environments and how does **JARVIS-1** tackle them. (Left) With situation-aware planning, **JARVIS-1** substantially improves the success rate on the challenging ObtainDiamond task, compared to the baseline (GPT) without it. Note: Due to resource constraints, we can only provide human results of 10-min gameplay; (Middle) As task complexity increases (STONE $\rightarrow$ IRON $\rightarrow$ DIAMOND), **JARVIS-1** exhibits more significant advantages thanks to interactive planning; (Right) Success rate gain (indicated by the color depth) on selected tasks (x-axis) given in-context experiences on other tasks (y-axis) retrieved from the multimodal memory. With life-long learning and memory, **JARVIS-1** can utilize prior experiences on relevant tasks for better planning.

II. CHALLENGES FOR OPEN-WORLD AGENTS

Compared to canonical scenarios with relatively small scale, simple dynamics, and limited tasks, open-world environments impose substantial challenges to building agents that can accomplish a diverse set of tasks [10], [11], [16], [18], [19], [24], [50]. In this section, we will review three major challenges we’ve identified during the development of **JARVIS-1**.

A. Challenge I: Situation-Aware Planning

In an open world, there could be various possible paths towards an open-world goal. However, not all of them are plausible or equally efficient given a certain *situation* (location, inventory status, etc.). For example, building a bed  can be done through collecting wool from sheeps , haunting spiders for strings , or trading with villagers . Depending on the current location and its proximity to these subjects, some options can be more viable and more efficient than others. Further, the agent’s own situation can also change throughout the episode, e.g. day and night shifts, weather conditions (bringing different types of danger), tool usage (it can be broken). To this end, the plan needs to be constantly updated based off the current situation. Fig. 2 (left) shows that when attempting the “ObtainDiamondPickaxe” task with a GPT-based planner that produces plans only at the beginning without looking at the current situation, the agent failed to complete the task as opposed to human players and **JARVIS-1**, which perform situation-aware planning from time to time. We’ve observed that many failures coming from this were attributed to the agent’s inability to adapt to the changing situations including entering a new biome, the tool being used becoming broken, etc.

B. Challenge II: Task Complexity

The second challenge comes from the higher task complexity in open-world environments. Due to the richness of terrains, objects, and action space, tasks in open-world domains usually require substantially long planning horizons as well as good accuracy and precision. For example, the task

ObtainEnchantingTable  includes more than 20 different sub-goals and therefore demand significantly longer reasoning steps. Meanwhile, many of these sub-goals have to be achieved precisely with the exact object name, quantities, and preconditions, e.g., mine 3 obsidian with diamond pickaxe, craft 1 diamond pickaxe from 3 diamonds and 2 sticks on crafting table; otherwise, the subsequent sub-goals won’t be executed due to unfulfilled preconditions. To tackle this, we may refer to some approaches in LLM reasoning, e.g. self-debugging [13] and turning the planning into an interactive fashion. In Fig. 2 (Middle), we’ve shown that as the complexity of the task increases, our **JARVIS-1**, which uses interactive planning [50] to mitigate the aforementioned issues (details can be found in Section III-B), elicits more significant advantages over the baseline (GPT) planner.

C. Challenge III: Life-Long Learning

Finally, being open world often implies offering an infinite number of tasks. Clearly, it is difficult for an agent to master all tasks or generalize to arbitrary tasks without additional learning. To this end, agents in an open world should be able to learn novel tasks while completing existing tasks, i.e. life-long learning. Furthermore, as many open-world agents employ large models [48], [50], [55], [60], canonical gradient-based learning could be extremely inefficient given the number of new tasks and experiences to learn. Our MLM-based **JARVIS-1** tackles this by adopting a memory to save all the experiences on past tasks. By retrieving memory entries relevant to the newly-coming task and putting them into the context as a reference, **JARVIS-1** is able to accumulate more experiences as the game continues and strengthen its own planning skills without gradient update. As illustrated in Fig. 2 (Right), for instance, both ObtainDiamondPickaxe  and ObtainDiamondAxe  require gathering almost identical materials. Therefore, they can help each other by using the experiences from the other task. Compared to completing these challenging tasks without any prior experiences, memory-based in-context life-long learning in **JARVIS-1** can bring significant advantages.

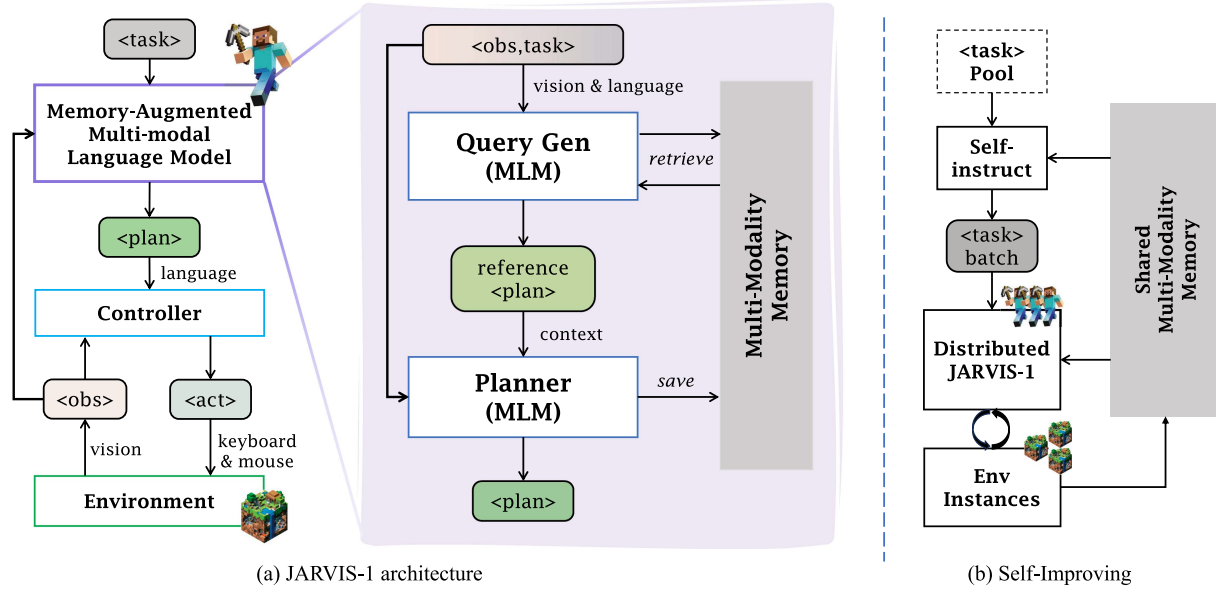


Fig. 3. Architecture of JARVIS-1 and its self-improving mechanism. (a) JARVIS-1 comprises a memory-augmented multimodal language model (MLM) that produces plans and a low-level action controller. JARVIS-1 also utilizes a multimodal memory to store and obtain experiences as references for planning. (b) JARVIS-1 can strengthen its own planning skills through exploration with its own proposed tasks (*self-instruct*) and a growing memory that helps with better planning on tasks that has been (partially) visited before.

III. MULTI-TASK AGENT WITH MEMORY-AUGMENTED MULTIMODAL LANGUAGE MODEL

This section details the architecture of the proposed JARVIS-1 agent. We begin with an overview of the modular agent design in Section III-A. Next, we elaborate on how to implement an interactive planning scheme with a multimodal language model, which helps with more accurate plans, especially on complex and long-horizon tasks in Section III-B. Finally, we show how to augment this planning framework with a multimodal memory to allow JARVIS-1 to strengthen its planning skill throughout the episode by in-context life-long learning in Sections III-C and III-D.

A. Overview

We aim to develop an agent capable of solving long-horizon instruction-following tasks using image observations and human-aligned actions. To accomplish this, we propose a multi-modal agent including an interactive **planner**, a goal-conditioned **controller**, and a **multimodal memory** of multimodal experiences. Upon receiving a task and the current observation, JARVIS-1 first utilizes the MLM to generate a multimodal query (**query gen**) that retrieves relevant planning experiences from the memory. These experiences will then be used along with the planning instruction to prompt the MLM-based planner. Leveraging its own pretrained knowledge as well as the retrieved reference plans, the planner will ultimately produce a series of K short-horizon goals $g_1, \dots, g_K$ to be executed by the controller. Once the plan is successfully executed, it will be stored in the memory along with the task and the agent situation when it was planned. We also empower JARVIS-1 with life-long learning by combining *self-instruct*, where JARVIS-1

will propose some tasks for itself to complete as a means of exploration; and *self-improve*, where multiple JARVIS-1 agents will be running in parallel to gather experiences, therefore helping with better planning later. We provide an illustration in Fig. 3.

B. Interactive Planning With MLM

As we have mentioned in Sections II-A and II-B, the primary challenges for planning in Minecraft come from the requirement of being able to plan for long-horizon tasks under dynamic observations. Confirmed by many prior arts [48], [50], [55], this makes it exceptionally hard to utilize canonical symbolic planners, which can be much less flexible. To this end, we take a multimodal language model (MLM) as zero-shot **planner** and combine it with an interactive planning framework to tackle these challenges.

Situation-aware planning with MLM: To achieve situation-aware planning, the planner must take the current observation into account, in addition to the task instruction [22], [54]. Specifically, we begin with translating the multimodal observation into text descriptions. As opposed to letting the MLM caption the scene directly, we first extract keywords of Minecraft items (e.g., “acacia tree”, “sheep”) from Minecraft wiki and utilizing GPT [9] to generate sentences that describe these observations. For example, a generated sentence could be “I can see sheep in the acacia plains”. Then the MLM will retrieve the condition sentence according to current visual observation during planning. Additional situation details including biome and inventory status are also converted into text using templates. Finally, we prompt the MLM again (the language part only) into a plan given the task instruction and all the aforementioned

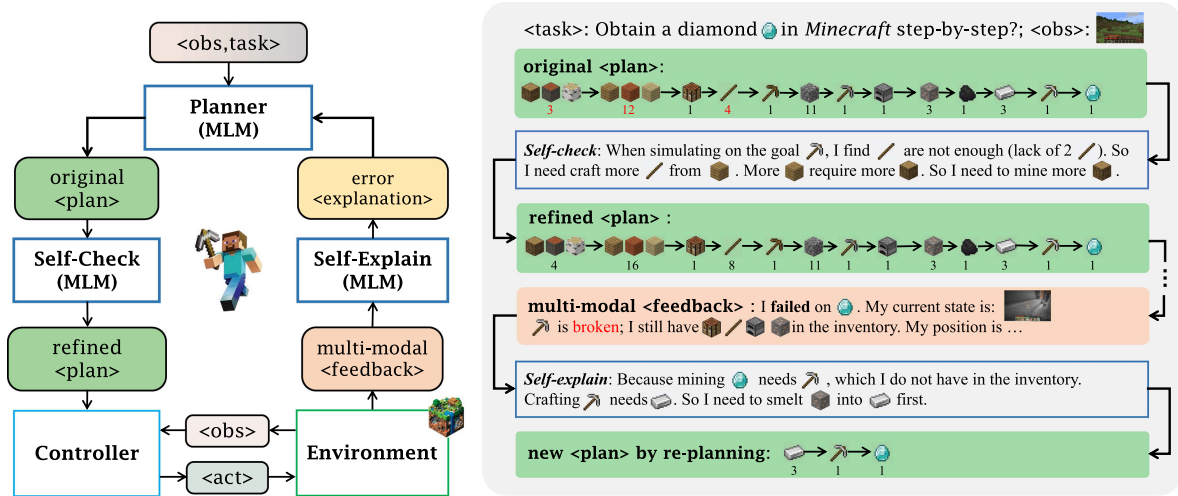


Fig. 4. Interactive planning in JARVIS-1. After receiving the current task instruction and observation, JARVIS-1 will produce an initial plan, which will go through *self-check* to get possible bugs (marked in red) fixed. Further, in case any error (also marked in red) occurs during the execution of the refined plan, JARVIS-1 will try to reason about the next move from the environmental feedback via *self-explain*. Interleaving self-check and self-explain significantly boosts the correctness and robustness of JARVIS-1 planning.

textual situation descriptions. Compared to end-to-end alternatives [6], [21], we find our composable usage of MLM provides higher quality situation descriptions and ultimately, plans with much less hallucination.

Planning with self-check: Our first layer of shield to ensure the correctness of plans involves *self-check*. Similar to self-debugging [13], given an initial plan, we ask JARVIS-1 to progressively simulate the plan execution, predict the resulting state after each step (primarily the state of inventory), and evaluate them. By verifying if these states satisfy the goal's precondition, JARVIS-1 can proactively identify potential plan flaws. Compared to the canonical planner where the agent has to encounter the error first before making a remedy, this upfront plan verification could mitigate the need for the agent to recover (re-plan) from more challenging situations due to plan failure. For instance, if an agent starts digging underground without sufficient wood, it would typically have to return to the surface, which substantially lowers the chance of completing the task.

Planning with environment feedback: Next, our interactive planning framework ventures into allowing JARVIS-1 to quickly recover from failure by leveraging environment feedback in a closed-loop fashion. The process is illustrated in Fig. 4. During plan execution, we feed the feedback to the MLM of JARVIS-1 in case there is any execution failure (possibly due to a flawed plan) and utilize its self-explain mechanism [44] to explain the error and locate the bugs in the original plan (we term this as *error explanation*). Finally, the MLM planner of JARVIS-1 will produce an improved plan based on both the outside environment feedback and the inside retrospective. Compared to other agents that rely on human intervention or privileged environment information [23], [60], JARVIS-1 has the ability to speculate about the reasons why current goals cannot be achieved, without the need for additional information or design.

C. Planning With Multimodal Memory in the Loop

To address the life-long learning challenge mentioned in Section II-C, we equip JARVIS-1 with multimodal memory to allow learning from its own past experiences. We will detail the formulation of the retrieval-augmented planning, query generation, and memory layout below.

Retrieval-augmented planning: Retrieval-augmented generation (RAG) [28], [38] enhances the quality of responses generated by LLMs by incorporating external sources of knowledge to complement the model's internal representation. We also utilize RAG to enhance JARVIS-1's long-term planning capability. Compared to official RAG methods leveraging the external knowledge library, we take the collected multimodal memory as the knowledge library and retrieve the interactive experiences as the demonstration prompt to augment the planning results. The formulation is as follows:

$$p(y | x) \approx \sum_{z \in \text{top-}k(p(\cdot | x))} p_{\eta}(z | x) p_{\theta}(y | x, z), \quad (1)$$

where x , y , and z denote instruction, plans, and retrieved memory entries respectively, and p_{η} and p_{θ} are denoted as retrieval and planning models. Such retrieval-augmented planning method helps JARVIS-1 ground the internal knowledge into the open-ended environments efficiently and leverage the historical interaction feedback to solve the hallucination within LLMs and produce more accurate plans.

Multimodal memory: We have demonstrated the layout of our multimodal memory on the right side of Fig. 5. From a high level, it is a key-value memory where the keys are multimodal, comprising both the task and the observation (or situation) made when this memory entry was created. The values are the plans that were successfully executed. Note that, since the plans in an open-world environment like Minecraft are situated (see Section II-A), there could be multiple entries that are with the same

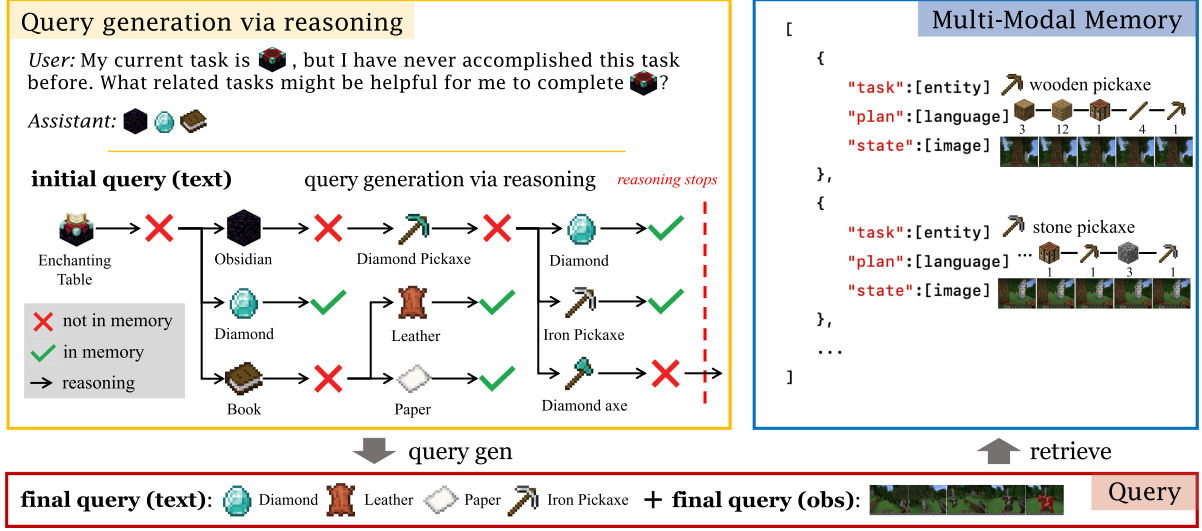


Fig. 5. Query generation in JARVIS-1. Given the current observation and the task, JARVIS-1 will first think backward and figure out the needed intermediate sub-goals. The reasoning will be bounded by a limited depth. The sub-goal that is present in the memory will join the current visual observation to form the final query. Entries that match the text query will be ranked by the perceiving distance of their states to the obs query and only the top entry of each sub-goal will be retrieved.

task but different observations and plans. As a result, JARVIS-1 needs to produce multimodal queries based on the current task and situations to retrieve the relevant memory entries.

Query generation via reasoning: When presented with an instruction as a task, we employ query generation via LLM reasoning to decompose the instruction into sub-tasks or related tasks, which will then be used as textual queries to retrieve relevant planning experiences as references for solving the current task. For instance, consider the instruction “craft 1 enchanting table with empty inventory” as shown in Fig. 5. JARVIS-1 queries the MLMs to identify the tasks that are required for achieving the main task in a backward search fashion, e.g., “obtain book  /diamond  /obsidian  with empty inventory”. The search depth is bounded for efficiency. Further, instead of relying solely on retrieval based on the text query [48], [60], we also propose to append the agent’s current visual observation to the textual query, resulting in a *multimodal query* to take the situation into account during memory retrieval.

Multimodal retrieval: After obtaining the textual and visual query, we compute the alignment between the query and each trajectory in multimodal memory. We first use the text encoder of CLIP model to compute the embedding of the query and task key of each entry in memory. We select the memory entries with similarity higher than the confidence threshold as the candidate entries. Then we will compute the visual state embedding of query and states in candidate entries. Then we sort the candidate entries with the visual embedding similarities, which can be formed as:

$$p_{\eta}(z | x) \propto \text{CLIP}_v(s_z)^{\top} \text{CLIP}_v(s_x), \quad (2)$$

where s_z and s_x are the visual key of memory entries and visual query, respectively. Finally, we retrieve the plan of top-k ($k=3$ in our experiments) candidate entries as reference prompt z .

D. Self-Improving Agents

Learning in Minecraft with memory: The remaining issue now is where the aforementioned multimodal memory comes from. Inspired by the life-long learning scheme in many close-world and open-world reinforcement learning problems [1], [2], [48], we propose the following learning approach for augmenting the memory in JARVIS-1: 1) First, we generate a set of tasks, which form some curricula for the agents to complete as means of exploration of the world. During this process, JARVIS-1 produces plans, interacts with the environment, embraces the errors, and stores all these experiences in the memory; 2) After this learning stage, we evaluate JARVIS-1 on various tasks. Therefore, JARVIS-1 is able to produce better plans with the memory teaming up with the planning experiences. In our experiments, we use this as the default setting for all tasks.

Exploration using self-instruct: The key issue to the success of learning with memory is how to effectively acquire useful experiences given a limited amount of time. We propose to use self-instruct [49] to generate the dynamic curriculum and guide JARVIS-1 to learn from the interactions with environments. In each round, we prompt the MLM to consider how capable JARVIS-1 is at this point and subsequently select tasks from a task pool to explore. We find that the curriculum almost follows the technical tree-growing direction. To accelerate the learning process, we augment the linear self-instruct to distributed learning in distributed environments with shared memory, i.e. speculative execution [27]. Specifically, we generate multiple executable tasks as candidate task batches and provide them to agents with the same memory for verification and execution in various different environments. Meanwhile, experiences are collected into a shared centralized memory. When all exploration tasks have been accomplished, we move to the next round, until the memory reaches a certain capacity.

Life-Long Learning: We’ve also observed that the aforementioned learning (where the memory is being filled) can be extended throughout the whole gameplay, where the agent gradually acquires more and more skills. As the gameplay continues, more and more experiences are pouring in, therefore **JARVIS-1** can find better references for challenging tasks like `ObtainDiamondPickaxe`, resulting in an improved success rate on these tasks. Further, there is no gradient update in this thanks to the memory-augmented MLM, i.e. we can do in-context life-long learning. In Section IV-D, we offer exploratory experiments to show the potential of such capability of **JARVIS-1**.

E. Implementation Details

Model Details in JARVIS -1 . Planning in Minecraft using a Multimodal Language Model requires the MLM to have a solid understanding of Minecraft-specific knowledge and visual perception capabilities. In the early stages of our research, we found that end-to-end Multimodal Language Models did not yet possess these capabilities. Therefore, we opted to combine an open-source visual model, MineCLIP [16] (which is trained on Minecraft image and text pair data from the internet), with a text-based language model, ChatGPT [40], to serve as the foundation model for JARVIS-1. In detail, we first scraped a large number of Minecraft demonstration images from the Minecraft Wiki. Using ChatGPT, we synthesized language descriptions for these images based on their alt-text labels. This process generated a library of 1000 image-text pairs. During JARVIS-1’s execution, we used MineCLIP’s vision encoder to compute the similarity between the current visual observation and the images in the library. We selected the top-3 most similar images, and their corresponding language descriptions were fed into ChatGPT to generate a caption for the current visual observation. This method effectively allowed us to enhance the Minecraft perception ability of the language-based ChatGPT. By combining the visual understanding from MineCLIP with the language generation capabilities of ChatGPT, we created a more capable MLM for JARVIS-1. To further conduct an ablation study on the model architecture used by JARVIS-1, we performed ablation experiments on the vision perception model and language model parts, as detailed in Section IV-C1 and Section IV-C3.

Controller: Tasks in Minecraft are usually related to mine and craft goals. The mine goals require the agent to collect raw materials from the environment using the appropriate tools. The craft goals ask the agent to use the recipe to generate new items with existing materials in inventory. The mine goals are achieved through STEVE-1 [30] with text condition during implementation. The environment can directly executes the craft and smelt actions (`craft/smelt` with argument), which are same as MineDojo [16].

Interactive Planner: **JARVIS-1** relies on the Multi-modal Language Model for planning, self-checking, and self-explaining, and can accept three types of inputs: visual images, language, and symbolic information (including inventory, located position, home, current life statistics, etc.). Specifically, this is a hybrid model with language processing capabilities derived from the GPT model [40]. The visual ability comes from

MineCLIP [16]. We collected approximately 1000 Minecraft text data from the internet and calculated the similarity between the current vision observation and these text data. The text above the similarity threshold (0.7 in practice) will be selected for the GPT model’s prompt. Symbolic information is converted into natural language text through a designed template. All modalities are ultimately captured as language and processed by the GPT model. Different modules in **JARVIS-1** (e.g. self-check and self-explain) are completed through MLM based on different prompts. The specific prompt design can be found in Appendix Section B.

Memory: Multimodal memory records every successful trajectory experience of **JARVIS-1**, including the task goals that the agent needs to execute, the actual goal sequence (plan) executed by the agent, and the state (visual observation and symbolic information returned from the environment) when the agent completes the task. In a specific implementation, memory is a list where each trajectory experience is encoded as a dictionary, including the keys `task`, `state`, and `plan`.

IV. EXPERIMENTS

In the experiments, our goal is to 1) evaluate the general performances of **JARVIS-1** on the challenging Minecraft tasks, especially on its advantages over baselines that do not (fully) address the aforementioned issues in open-world agents; 2) understand the factors that contributes to the general results; 3) explore the potential of **JARVIS-1** in terms of life-long learning and its benefits to long-horizon tasks. To this end, we will first briefly introduce the evaluation settings, then cover the main comparative results and ablation studies, and conclude with an exploratory trial on long-horizon tasks.

A. Experimental Setups

We evaluate **JARVIS-1** in Minecraft, with tasks selected from the recently introduced Minecraft Universe Benchmark [31]. For the reader’s convenience, we provide details on the basic setups below.

Environment setting: To ensure realistic gameplay, the agent needs to utilize observation and action spaces that are similar to those used by humans. Instead of manually designing a custom interface for models to interact with the environment, as done in previous methods such as MineDojo [16], GITM [60], and Voyager [48], we opt for using the native human interface provided by Minecraft. This applies to both the observation and action space. The model operates at a speed of 20 frames per second and is required to use a mouse and keyboard interface when interacting with human GUIs. For more information on the detailed descriptions of the observation and action spaces, please refer to the Appendix.

Task setting: In Minecraft, players have access to thousands of items, each with specific acquisition requirements or recipes. For example, stone-type items can only be obtained using a pickaxe, and two planks can be crafted into four sticks (these requirements are available on the Minecraft Wiki¹). In survival mode, players must obtain each type of item from the

¹https://minecraft.fandom.com/wiki/Minecraft_Wiki

TABLE I
CHARACTERISTICS OF 11 TASK GROUPS ENCOMPASSING OVER 200 MINECRAFT TASKS

Group	Task Num.	Max. Steps	Initial Inventory	Biome	Language Instruction
Wood	34	12k	null	Plains/Forest	Pick up a <code>wooden_pickaxe</code> .
Wood-Variants	43	12k	null	Savanna/Jungle/Taiga	Pick up a <code>acacia_boat</code> .
Stone	10	12k	<code>iron_axe</code>	Plains/Forest	Craft a <code>furnace</code> given an iron axe.
Iron	22	12k	<code>iron_axe</code>	Plains/Forest	Smelt and craft an <code>iron_door</code> given an iron axe.
Gold	9	36k	<code>iron_axe</code>	Plains/Forest	Smelt and craft an <code>golden_axe</code> given an iron axe.
Diamond	7	36k	<code>iron_axe</code>	Plains/Forest	Dig down to mine diamond and craft <code>diamond_pickaxe</code> .
Redstone	7	36k	<code>iron_axe</code>	Plains/Forest	Mine redstone and make <code>dropper</code> given an iron axe.
Blocks	15	12-36k	<code>iron_axe</code>	Plains/Forest	Dig down to mine <code>lapis_lazuli</code> block.
Armor	17	12-36k	<code>iron_axe</code>	Plains/Forest	Craft <code>diamond_boots</code> given an iron axe and equip it.
Decoration	17	12k	<code>iron_axe</code>	Flower Forest	Obtain the <code>bed</code> and dye it <code>red</code> .
Food	9	12k	<code>iron_axe</code>	Plains	Kill sheep to obtain <code>mutton</code> and cook it.

environment or craft/smelt the object item from materials. We choose over 200 tasks from the Minecraft Universe Benchmark [31] for evaluation. These tasks are related to items that can be obtained in the Minecraft overworld. For the convenience of statistics, we have classified them into 11 groups according to recommended categories² in Minecraft (see Table I). Due to the varying complexity of these tasks, we adopt different maximum gameplay durations (Max. Steps) for each task. The limit is determined by the average time the human players need to accomplish the corresponding task. Other details about each task, such as language instruction, maximum steps, evaluation times, biome, and initial inventory when the agent is born into the world can be found in Appendix Tables 7–16.

Evaluation metrics: By default, the agent always starts in survival mode, with an empty inventory. A task is considered a success when the target object is obtained within a specified time. Due to the open-world nature of Minecraft, the world and initial position that the agent is spawned at could vary a lot. Therefore, we conducted at least 30 tests for each task using different seeds and reported the average success rate to ensure a thorough assessment. Further, since we categorize the tasks into groups, we also report mean and variance values for each group for ease of presentation.

B. Main Results

We compare JARVIS-1 with other multi-task instruction-following agents based on LLM, including Instruct GPT [22], [41], ReAct [54], Inner Monologue [23], DEPS [50]. Since some methods are not originally experimented in Minecraft, we reproduce them to conform to the Minecraft specification based on prompt and feedback template design. All LLM-based methods access the LLM model through OpenAI API. And all hyper-parameters of LLM including temperature are kept as default.

The average success rates for every task group are listed in Table II. JARVIS-1 achieves the best performance with all meta tasks. It is important to note that in Minecraft, the technology

tree can be formed by Group Wood, Stone, Iron, Gold, and Diamond. The tasks become increasingly difficult as you progress through the tree. For more difficult tasks such as obtaining a gold ingot or a diamond, the agents typically need to perform more actions and longer goal sequences in order to complete the task. As a result, the success rate of all agents decreases as the difficulty level increases. It is evident that reasoning methods (ReAct [54] vs. GPT [22], [41]) and interactive re-planning with feedback (Inner Monologue [23] vs. GPT) effectively enhance the agent’s task performance in an open world. However, these approaches still face challenges when dealing with long-horizon tasks, specifically in the Iron and Diamond group. DEPS [50], on the other hand, enables agents to accomplish diamond-related tasks through interactive long-horizon planning accompanied by descriptions and explanations. Nevertheless, its reliability remains very low at approximately 2.5%.

In comparison to DEPS [50] without memory, JARVIS-1 demonstrates superior performance even in challenging tasks due to its extensive experience. In diamond-related tasks specifically, the success rate has increased by nearly 3 times (8.99% vs 2.42%). And JARVIS-1 usually only requires 2-3 rounds of re-planning to generate the correct executable plan, whereas DEPS requires more than 6 rounds. This means that JARVIS-1 saves a significant amount of LLM tokens and thinking time, enabling more efficient plan execution and providing additional steps and tokens for handling uncertainty in the environment.

Based on our observations, we have found that the bottleneck for JARVIS-1 in tasks involving diamonds often lies with the Controller’s inability to perfectly execute short-horizon text instructions generated by LLM. Therefore, it is worth exploring methods for generating plans that are easier for the controller to execute or improving the controller’s ability to follow instructions.

C. Ablation Studies

1) *JARVIS-1 based on different LMs:* We conducted ablation experiments on various Language Models, including OpenAI’s ChatGPT [41] and GPT-4 [40]. Among these models, GPT-4 has more parameters and has been proven to outperform

²<https://minecraft.fandom.com/wiki/Tutorials/Organization#Categories>

TABLE II
RESULTS OF JARVIS-1 AND BASELINES ON MINECRAFT

Group	Task	GPT [22]	ReAct [54]	Inner Monologue [23]	DEPS [50]	JARVIS-1
Wood		26.67	45.00	36.67	75.00	91.55
	AVG	27.30±14.86	40.31±13.30	60.15±19.41	80.23±17.32	88.84±16.82
Wood Var		6.67	36.67	30.00	36.67	60.47
	AVG	24.39±11.08	38.13±12.81	53.39±12.86	68.75±12.32	76.78±12.27
Stone		20.00	20.00	66.67	75.00	94.20
	AVG	20.21±12.32	39.00±12.15	52.86±16.90	69.27±7.78	88.69±4.87
Iron		0.00	0.00	3.33	20.00	33.82
		3.33	6.67	0.00	20.00	38.10
	AVG	3.27±2.85	4.61±3.63	5.20±5.17	16.92±4.69	34.63±10.61
Gold		0.00	2.00	2.00	6.00	14.49
	AVG	0.00±0.00	0.45±0.60	0.59±0.64	2.20±1.55	6.85±4.71
Diamond		0.00	0.00	1.00	2.00	9.20
		0.00	0.00	0.00	2.50	6.22
	AVG	0.00±0.00	0.35±0.48	0.96±0.67	2.42±1.01	8.99±2.68
Redstone		0.00	2.00	0.00	10.00	22.78
	AVG	1.04±1.30	1.14±1.18	0.69±1.68	6.02±3.61	17.51±9.34
Blocks		16.67	33.33	43.33	53.33	86.67
	AVG	45.64±33.88	49.35±30.51	55.71±29.43	58.02±27.68	80.34±21.09
Armor		6.67	0.00	10.00	10.00	30.30
	AVG	1.36±2.25	0.50±0.88	3.10±4.71	3.71±3.78	13.44±14.62
Decoration		15.00	15.00	15.00	25.00	50.00
	AVG	17.12±11.59	17.13±9.19	12.03±10.19	29.59±15.94	46.67±23.39
Food		13.33	16.67	25.00	16.67	43.55
	AVG	9.40±4.29	15.56±6.83	20.78±11.99	22.85±8.15	46.75±11.16

The detailed task instructions, settings and results can be found in the appendix.

ChatGPT in extensive research [48]. We also select the open-source pre-trained LLaMA2 70B model [47]. Additionally, we gathered a substantial amount of Minecraft-related text from the internet as training data and further fine-tuned LLaMA2 13B. The experiments were conducted on a subset of Minecraft tasks using different language models. Each JARVIS-1 learns for 4 epochs of interaction with all task sets and evaluates on task subset across at least 20 seeds. The experimental results are presented in Fig. 7.

Table 7 demonstrates that ChatGPT, despite having fewer parameters, achieves nearly identical success rates as GPT-4. This suggests that language models equipped with memory can significantly enhance planning abilities. In Minecraft-related tasks, the open-source pre-trained LLaMA2 70B exhibits a notable performance gap compared to OpenAI models, particularly in long-horizon tasks. However, by finetuning LLaMA2 with fewer parameters, its performance on Minecraft tasks improves substantially. This indicates that the open-source model lacks

knowledge specific to Minecraft and requires further finetuning for the successful completion of such tasks.

2) *Ablation on Memory*: We also conduct ablation experiments on the multimodality memory and retrieval methods. We set JARVIS-1 w/o memory module as the baseline agent. We first evaluate JARVIS-1's performance with different memory sizes (representing different learning stages) as shown in Fig. 6, which demonstrates the effectiveness of self-improving within JARVIS-1. We further conduct the experiments on a subset of Minecraft tasks using three different retrieval methods: retrieval with textual instruction embedding only (Text Memory), synergizing reasoning and retrieval with text embedding (Text Memory+Reasoning), and synergizing reasoning and retrieval with multimodality embedding (Multimodal Memory+Reasoning). Except for the memory and retrieval methods, all others are kept the same. The results are listed in Fig. 8.

The experiments show that reasoning before retrieval can effectively improve retrieval accuracy. Retrieval based on a

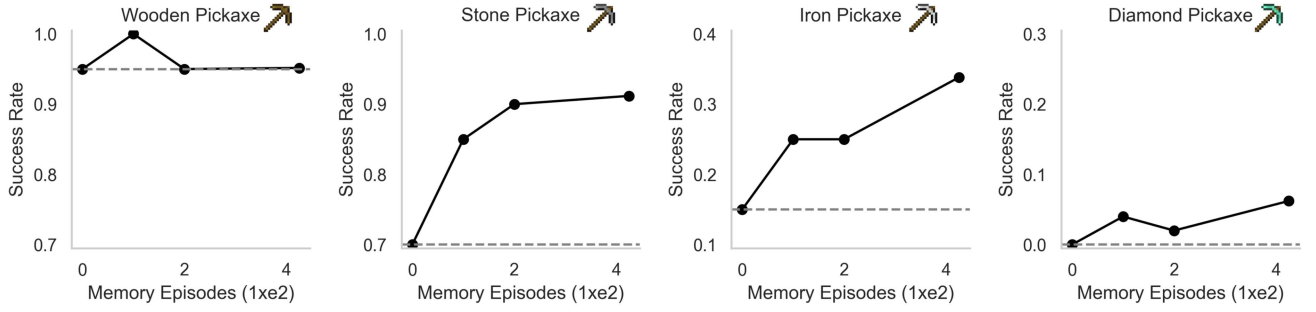


Fig. 6. Success rate by memory size for different items. We evaluated the performance of JARVIS-1 at different memory sizes (representing different learning stages) by measuring the success rate (% Episodes) of completing key items on the Minecraft technology tree. As the learning progressed, we observed an improvement in completion rates for all items, with an increasing number of successful trajectories being included in memory. After 4 epochs of learning, JARVIS-1 had accumulated a total of 425 successful trajectories in its memory.

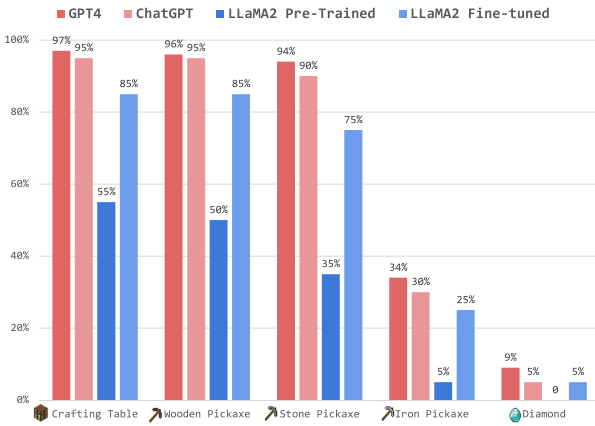


Fig. 7. Success rates for different language models on Minecraft tasks. We found open-sourced LLaMA2-70B models [47] lack knowledge related to Minecraft, so the pre-trained model performs poorly. We further finetuned the LLaMA2-13B model on a Minecraft text dataset collected from the internet, and it shows performance similar to ChatGPT on Minecraft.

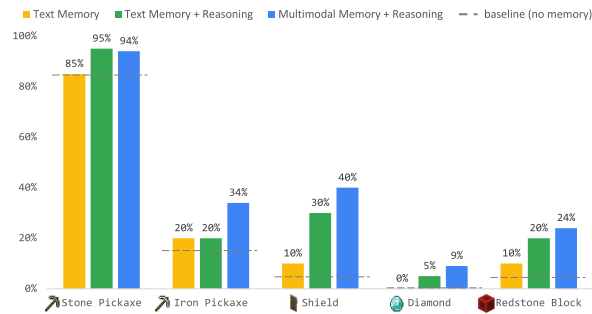


Fig. 8. Success rates for different retrieval methods with memory on Minecraft tasks. JARVIS-1, which synergizes reasoning and retrieval with multimodal memory, achieves best.

multimodal state including vision observation and symbolic information (e.g., inventory, location, etc) is better than only considering the text embedding.

3) *Ablation on Vision Perception Methods:* As the research community developed more advanced end-to-end Vision-Language Models, such as OpenAI’s GPT-4V [40] and open-source models like Llava [35], ShareGPT(ShareCaptioner) [12],

TABLE III
ABLATION EXPERIMENTS ON VARIOUS VISION PERCEPTION MODELS IN JARVIS-1

Model								
MineCLIP [16]	0.90	0.95	0.40	0.15	0.10	0.45	0.30	0.50
ShareGPT [12]	0.95	0.90	0.40	0.15	0.10	0.25	0.25	0.35
GPT4-V [40]	1.00	1.00	0.50	0.15	0.10	0.50	0.30	0.55

and Fuyu [5], we conducted ablation experiments to explore the effectiveness of different vision encoders in the MLM. For these experiments, we standardized the language model to GPT-4 to ensure fairness. The results of using different visual perception methods in JARVIS-1 across various tasks are summarized in Table III. As seen in the table, GPT-4 V provided the best overall performance across the tasks, indicating the advantage of end-to-end Vision-Language Models. However, MineCLIP also performed robustly, validating our original design choice.

D. Long-Horizon Challenges

Most concurrent multi-task agents in Minecraft can only handle short-term tasks and struggle with long-horizon tasks like CraftingDiamondPickaxe. The VPT foundation model [4] is capable of accomplishing various tasks in Minecraft but lacks the ability to execute human instructions. To address this limitation, Reinforcement Learning is required to fine-tune the VPT foundation model for specific task completion. However, after fine-tuning, VPT may experience a decline in performance for other tasks while focusing on the specified task. In contrast, Steve-1 [30] has implemented goal-conditioned fine-tuning on VPT-early-game, enabling it to follow human text instructions while maintaining multitasking capabilities. However, Steve-1 primarily focuses on low-level tasks like obtaining dirt, collecting flowers and chopping trees. When it comes to long-horizon tasks such as starting from scratch by obtaining a wooden pickaxe, Steve-1 still encounters difficulties.

DEPS [50] also utilizes LLM as a planner, but it lacks the ability to learn from experience in different tasks and apply that knowledge to new ones. Additionally, DEPS is limited in its re-planning rounds due to the LM’s context constraints. The

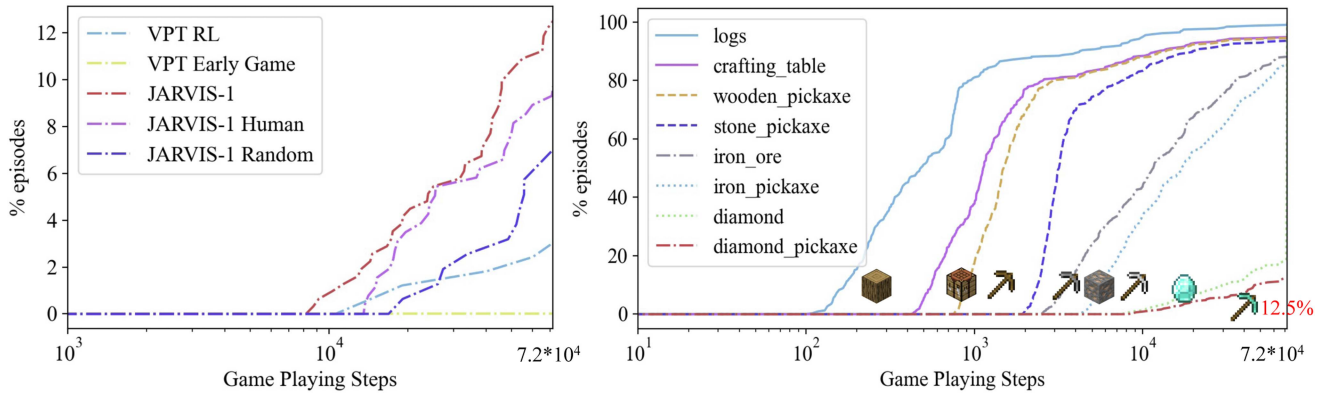


Fig. 9. (Left) The success rate of different models in the ObtainDiamondPickaxe challenge over gameplay time. VPT RL is finetuned from VPT early game with reinforcement learning over 1.4 million episodes. JARVIS-1 agent and its variants have interacted with Minecraft for over 4 epochs on all tasks in the task pool. Typically, it takes a skilled person over 20 minutes (24,000 steps) to obtain a diamond pickaxe. (Right) The success rate of obtaining important intermediate items during the process of synthesizing a diamond pickaxe of JARVIS-1. This task has been evaluated over 300 times on different seeds. These curves indicate that as the game progresses, the success rates of obtaining all intermediate items are increasing, which indicates that JARVIS-1 is constantly improving its skills.

experiments reveal that DEPS has a success rate of less than 50% in generating accurate and executable plans for acquiring diamonds. The probability of DEPS successfully obtaining diamonds in the environment is approximately 0.59%. Consequently, DEPS continues to face challenges when attempting to finish long-horizon tasks within the Minecraft world.

Even human players who have mastered the distribution pattern of diamonds achieve success rates of obtaining diamonds and crafting a diamond pickaxe (which requires at least three diamonds) within 10 minutes at approximately 15% and 12%, respectively. JARVIS-1 performs better in the ObtainDiamondPickaxe challenge. Compared to the state-of-the-art model, which has undergone RL-finetuned VPT, JARVIS-1 has more than doubled the success rate of obtaining a diamond pickaxe (6.22% vs 2.5% within 20 minutes).

To increase the chances of obtaining diamonds, we extended the game playing time to 60 minutes (72000 game-playing steps, as shown in Fig. 9). As a result, JARVIS-1's success rate in acquiring a diamond pickaxe improved from 6.2% to 12.5%. The graph on the right side of Fig. 9 illustrates how the success rate of intermediate milestone items changes over time, indicating that JARVIS-1 tends to improve with longer game-playing time. We also conduct two variants of JARVIS-1 with different self-improving curriculum: human-written and random-generated. All three JARVIS-1 have collect experiences into memory with the curriculum for 4 epochs before evaluation in 60 minutes. The results show that JARVIS-1 with GPT-generated curriculum can finish the task within the shortest game-playing steps and achieve the best performance in 60 minutes.

In contrast, VPT's success rate barely changed when we increased the time from 20 minutes to 60 minutes (from 2.5% to 3%). This can be attributed to Minecraft's durability system where prolonged underground exploration often leads to pickaxe damage. When JARVIS-1's pickaxe breaks, it dynamically re-plans based on its current inventory and crafts a new one. However, VPT-RL exhibits perplexing behaviors at this stage by using inappropriate tools for mining stone or crafting unnecessary items. This comparison demonstrates that JARVIS-1

possesses superior generalization and planning abilities for long-horizon tasks.

Note that our method is designed to be multi-task in its nature and not finetuned through imitation learning on specific dataset or reinforcement learning.

V. RELATED WORKS

A. LLM-Based Agents

There have been some methods leveraging the large language model to generate actions for high-level tasks in embodied environments [14], [17], [34], [36], [56], [57], [58]. [22] decompose natural language commands into sequences of executable actions by text completion and semantic translation, while SayCan generates feasible plans for robots by jointly decoding an LLM weighted by skill affordances from value functions [8]. Some methods also leverage the LLM to produce the program code as a plan for better execution [29], [32], [45]. However, the above methods assume that the initial plan from the LLM is correct. When there are bugs in the initial plan, it's difficult for the agent to finish the task successfully. Recent research frequently employs LLM as an interactive planner, harnessing its self-updating capabilities to enhance the plan's executability over time [44], [46], [50]. Inner Monologue [23] pilots the front of interactive planning with LLMs, which introduces the feedback (including success detection and scene description) to the planner. However, we found it could still suffer from accumulative planning errors, especially in long-horizon open-world tasks. ReAct [54] will reason about the agent state before acting, which indicates that various reasoning methods [51], [52], [53] are beneficial for planning. LLM-based planning methods often use the fixed pretrained LLM as the agent, while we focus more on life-long and continual learning for agents in open-world environments [25], [26], [48]. For better leveraging historical interaction between agent and environments, an explicit memory [42], [60] for more historical chatting has been leveraged for bigger storage of agent experiences. However, the above methods usually rely only on a text-based environment and

struggle to execute plans in partial-observed visual open-world environments.

B. Minecraft Agents

Developing generally capable agents in Minecraft to solve open-world tasks has gained increasing interest [4], [10], [11], [15], [16], [55], [58], [60]. As an early attempt, [39] studied task generalization in a simple Minecraft environment variant. It designed a two-stage pipeline, first mastering the prerequisite skills with parameterization trick, and then learning a metacontroller to execute the instructions. Moving to solve complex long-horizon tasks in Minecraft, works [33], [37], [39] explored the hierarchical architecture. In recent years, influenced by the trend of large-scale pre-training paradigms, a group of researchers have emerged, who are utilizing vast amounts of internet knowledge to train intelligent agents. [16] trained a visual-semantic alignment model, MineCLIP, using the correspondences between subtitles and video snippets available on YouTube, and used it to generate intrinsic rewards to guide policy learning. [4] utilizes a pre-trained inverse dynamics model to label actions in YouTube videos which are used to learn a foundation policy VPT through imitation learning. By bridging MineCLIP and VPT, [30] creates a performant instruction-following policy Steve-1 to solve open-world short-horizon tasks using hindsight relabeling and unCLIP tricks. However, Steve-1 can not solve complicated process-oriented tasks due to the expressive capability of its goal space. [11] learns to follow reference videos as the instruction by merely watching gameplay videos, which improves the capacity of goal space and reduces the cost of policy training. All of these methods focus on improving the smoothness and robustness of interaction between policy and environment. Inspired by the powerful language understanding and reasoning capabilities of large language models, researchers have begun to build Minecraft agents based on LLMs. [48] used LLM to guide the agent to explore the Minecraft world by acquiring diverse skills, making novel discoveries, and generating goal proposals. [60] integrated LLM with text-based knowledge and memory to equip the agent with common sense and past experiences for higher reasoning efficiency. [55] used LLM to guide the agent to explore the Minecraft world and interact with the environment with reinforcement learning control policies.

VI. CONCLUSION

We propose a multi-task agent JARVIS-1 designed for the complex environment of Minecraft, which marks a significant advancement in achieving human-like planning within an open-world setting. By leveraging pre-trained Multi-modal Language Models, JARVIS-1 not only effectively interprets multimodal inputs but also adeptly translates them into actions. Its integration of a multimodal memory, which draws from both ingrained knowledge and real-time game experiences, enhances its decision-making capabilities. The empirical evidence of its prowess is evident in its impressive performance across a wide array of tasks in Minecraft. Notably, its achievement in the long-horizon diamond pickaxe task, where it achieved a completion rate that surpasses VPT-RL by up to five times, underscores its

potential and the strides made in this domain. This breakthrough sets the stage for the future of more versatile and adaptable agents in complex virtual environments.

ACKNOWLEDGMENT

The authors sincerely thank Dr. Rita Zhang, Zhixiang Dai at NVIDIA for their valuable technical support of GPU computing.

REFERENCES

- [1] D. Abel, D. Arumugam, L. Lehnert, and M. Littman, "State abstractions for lifelong reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 10–19.
- [2] D. Abel, Y. Jinnai, S. Y. Guo, G. Konidaris, and M. Littman, "Policy and value transfer in lifelong reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 20–29.
- [3] J.-B. Alayrac et al., "Flamingo: A visual language model for few-shot learning," 2022, *arXiv:2204.14198*.
- [4] B. Baker et al., "Video pretraining (VPT): Learning to act by watching unlabeled online videos," 2022, *arXiv:2206.11795*.
- [5] R. Bavishi et al., "FUYU-8B: Introducing our multimodal models," 2023. [Online]. Available: <https://www.adept.ai/blog/fuyu-8b>
- [6] A. Brohan et al., "RT-2: Vision-language-action models transfer web knowledge to robotic control," 2023, *arXiv:2307.15818*.
- [7] A. Brohan et al., "Rt-1: Robotics transformer for real-world control at scale," 2022, *arXiv:2212.06817*.
- [8] A. Brohan et al., "Do as I can, not as I say: Grounding language in robotic affordances," in *Proc. 6th Annu. Conf. Robot Learn.*, 2022.
- [9] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 1877–1901.
- [10] S. Cai, Z. Wang, X. Ma, A. Liu, and Y. Liang, "Open-world multi-task control through goal-aware representation learning and adaptive horizon prediction," 2023, *arXiv:2301.10034*.
- [11] S. Cai et al., "Groot: Learning to follow instructions by watching gameplay videos," 2023, *arXiv:2310.08235*.
- [12] L. Chen et al., "ShareGPT4V: Improving large multi-modal models with better captions," 2023, *arXiv:2311.12793*.
- [13] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," 2023, *arXiv:2304.05128*.
- [14] I. Dasgupta et al., "Collaborating with language models for embodied reasoning," in *Proc. NeurIPS Found. Models Decis. Mak. Workshop*, 2022.
- [15] Z. Ding, H. Luo, K. Li, J. Yue, T. Huang, and Z. Lu, "Clip4mc: An RL-friendly vision-language model for minecraft," 2023, *arXiv:2303.10571*.
- [16] L. Fan et al., "Building open-ended embodied agents with internet-scale knowledge," in *Proc. Adv. Neural Inf. Process. Syst. Datasets Benchmarks*, 2022.
- [17] R. Gong et al., "Mindagent: Emergent gaming interaction," 2023, *arXiv:2309.09971*.
- [18] W. H. Guss et al., "The mineRL 2020 competition on sample efficient reinforcement learning using human priors," 2021, *arXiv:2101.11071*.
- [19] W. H. Guss et al., "Neurips 2019 competition: The minerl competition on sample efficient reinforcement learning using human priors," 2019, *arXiv:1904.10079*.
- [20] W. H. Guss et al., "MineRL: A large-scale dataset of minecraft demonstrations," 2019, *arXiv:1907.13440*.
- [21] J. Huang et al., "An embodied generalist agent in 3D world," 2023, *arXiv:2311.12871*.
- [22] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, "Language models as zero-shot planners: Extracting actionable knowledge for embodied agents," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 9118–9147.
- [23] W. Huang et al., "Inner monologue: Embodied reasoning through planning with language models," 2022, *arXiv:2207.05608*.
- [24] A. Kanervisto et al., "Minerl diamond 2021 competition: Overview, results, and lessons learned," in *Proc. Neural Inf. Process. Syst.*, 2022.
- [25] Z. Ke, H. Lin, Y. Shao, H. Xu, L. Shu, and B. Liu, "Continual training of language models for few-shot learning," 2022, *arXiv:2210.05549*.
- [26] Z. Ke, Y. Shao, H. Lin, T. Konishi, G. Kim, and B. Liu, "Continual pre-training of language models," in *Proc. 11th Int. Conf. Learn. Representations*, 2022.
- [27] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 19274–19286.

- [28] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 9459–9474.
- [29] J. Liang et al., "Code as policies: Language model programs for embodied control," 2022, *arXiv:2209.07753*.
- [30] S. Lifshitz, K. Paster, H. Chan, J. Ba, and S. McIlraith, "Steve-1: A generative model for text-to-behavior in minecraft," 2023, *arXiv:2306.00937*.
- [31] H. Lin, Z. Wang, J. Ma, and Y. Liang, "MCU: A task-centric framework for open-ended agent evaluation in minecraft," 2023, *arXiv:2310.08367*.
- [32] K. Lin, C. Agia, T. Migimatsu, M. Pavone, and J. Bohg, "Text2Motion: From natural language instructions to feasible plans," 2023, *arXiv:2303.12153*.
- [33] Z. Lin, J. Li, J. Shi, D. Ye, Q. Fu, and W. Yang, "Juewu-MC: Playing minecraft with sample-efficient hierarchical reinforcement learning," 2021, *arXiv:2112.04907*.
- [34] B. Liu et al., "LLM+P: Empowering large language models with optimal planning proficiency," 2023, *arXiv:2304.11477*.
- [35] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024.
- [36] J. Mai, J. Chen, B. Li, G. Qian, M. Elhoseiny, and B. Ghanem, "LLM as a robotic brain: Unifying egocentric memory and control," 2023, *arXiv:2304.09349*.
- [37] H. Mao et al., "SEIHAI: A sample-efficient hierarchical ai for the minerl competition," in *Proc. Distrib. Artif. Intell.: 3rd Int. Conf.*, Shanghai, China, Springer, 2021, pp. 38–51.
- [38] Y. Mao et al., "Generation-augmented retrieval for open-domain question answering," 2020, *arXiv:2009.08553*.
- [39] J. Oh, S. Singh, H. Lee, and P. Kohli, "Zero-shot task generalization with multi-task deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 2661–2670.
- [40] OpenAI. Gpt-4 technical report, 2023.
- [41] L. Ouyang et al., "Training language models to follow instructions with human feedback," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 27730–27744.
- [42] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," 2023, *arXiv:2304.03442*.
- [43] S. Reed et al., "A generalist agent," 2022, *arXiv:2205.06175*.
- [44] N. Shinn, B. Labash, and A. Gopinath, "Reflexion: An autonomous agent with dynamic memory and self-reflection," 2023, *arXiv:2303.11366*.
- [45] I. Singh et al., "ProgPrompt: Generating situated robot task plans using large language models," 2022, *arXiv:2209.11302*.
- [46] H. Sun, Y. Zhuang, L. Kong, B. Dai, and C. Zhang, "Adaplaner: Adaptive planning from feedback with language models," 2023, *arXiv:2305.16653*.
- [47] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.
- [48] G. Wang et al., "Voyager: An open-ended embodied agent with large language models," 2023, *arXiv:2305.16291*.
- [49] Y. Wang et al., "Self-instruct: Aligning language models with self-generated instructions," in *Proc. 61st Annu. Meeting Assoc. Comput. Linguistics*, 2023, pp. 13484–13508.
- [50] Z. Wang, S. Cai, A. Liu, X. Ma, and Y. Liang, "Describe, explain, plan and select: Interactive planning with large language models enables open-world multi-task agents," 2023, *arXiv:2302.01560*.
- [51] J. Wei et al., "Chain of thought prompting elicits reasoning in large language models," in *Proc. 36th Conf. Neural Inf. Process. Syst.*, 2022, pp. 24824–2483.
- [52] Y. Wu et al., "SPRING: GPT-4 out-performs RL algorithms by studying papers and reasoning," 2023, *arXiv:2305.15486*.
- [53] S. Yao et al., "Tree of thoughts: Deliberate problem solving with large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, vol. 36.
- [54] S. Yao et al., "React: Synergizing reasoning and acting in language models," 2022, *arXiv:2210.03629*.
- [55] H. Yuan et al., "PLAN4MC: Skill reinforcement learning and planning for open-world minecraft tasks," 2023, *arXiv:2303.16563*.
- [56] A. Zeng et al., "Socratic models: Composing zero-shot multimodal reasoning with language," 2022, *arXiv:2204.00598*.
- [57] C. Zhang et al., "ProAgent: Building proactive cooperative AI with large language models," 2023, *arXiv:2308.11339*.
- [58] W. Zhang and Z. Lu, "Rladapter: Bridging large language models to reinforcement learning in open worlds," 2023, *arXiv:2309.17176*.
- [59] H. Zhao et al., "MMICL: Empowering vision-language model with multimodal in-context learning," 2023, *arXiv:2309.07915*.
- [60] X. Zhu et al., "Ghost in the minecraft: Generally capable agents for open-world environments via large language models with text-based knowledge and memory," 2023, *arXiv:2305.17144*.



more than 10 papers in top conferences and journals in machine learning such as *IEEE Transactions on Pattern Analysis and Machine Intelligence*, CVPR, ICML, ICLR, and NeurIPS, with 6 as the first author. He has received honors such as best paper with the ICML 2023 Workshop, ICLR Spotlight, AAAI oral presentation.



Shaofei Cai received the BEng degree from the School of Software Engineering, Xi'an Jiaotong University, in 2019, and the MEng degree from the Department of Computer Science and Technology, University of Chinese Academy of Sciences, in 2022. He is currently working toward the PhD degree with the School of Intelligence Science and Technology, Peking University. His research interests predominantly lie in the fields of computer vision, decision-making, reinforcement learning, and open-world agents.



Anji Liu is currently working toward the PhD degree with the Department of Computer Science, University of California, Los Angeles (UCLA). His primary research focus is deep generative models (diffusion models, probabilistic circuits, and variational autoencoders). Other than understanding and mitigating the fundamental challenges toward good modeling performance, he is especially interested in efficient exact/approximate inference with guarantees of various deep generative models from both theoretical perspectives and empirical perspectives.



Yonggang Jin received the bachelor's degree from Zhejiang University, in 2022. He is currently working toward the master's degree with the Beijing University of Posts and Telecommunications, under the supervision of professor Zhao Feng He. His research focuses on Large Language Models and Reinforcement Learning.



Jinbing Hou received the BS degree and the MS degree from the School of Computer Science, Beijing University of Posts and Telecommunications. He is currently a machine learning engineer with Polixir Technologies Co., Ltd. His research interests include large language models, behavior cloning, and compositional generalization.



Bowei Zhang is currently working toward the undergraduate degree in the School of Electronics Engineering and Computer Science, Peking University. His research interests include sequential decision-making and computer vision.



Yaodong Yang is a Boya assistant professor with the Peking University Institute for Artificial Intelligence, and programme director for LLM Safety Centre at the Beijing Academy of Artificial Intelligence (BAAI). Dr. Yang's research focuses on human-AI safe interaction and value alignment, covering areas such as reinforcement learning, AI alignment, multi-agent learning, and embodied AI. He has maintained a track record of more than 100 publications at top conferences and journals (*Nature Machine Intelligence*, *Artificial Intelligence*, *Journal of Machine Learning Research*, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *National Science Review*), with 6000+ Google Citations. He has been awarded the Best Paper Award Initial List with ICCV'23, the Best System Paper Award with CoRL'20, the Best Blue-Sky Paper Award with AAMAS'21, the Rising Star Award of ACM SIGAI China and World AI Conference 2022.

Haowei Lin biography and photograph not available at the time of publication.



Zhaofeng He received the PhD degree in pattern recognition and intelligent system from the Institute of Automation, Chinese Academy of Sciences (CASIA), in 2010. Now he serves as a professor in Beijing University of Posts and Telecommunications (BUPT), and is the founder of the Laboratory of Visual Computing and Intelligent System (VCIS). He has authored several top and international conference and journal papers in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, CVPR, etc. His research interests include biometrics, computer vision,

intelligent system.



Xiaojian Ma received the bachelor's degree in computer science from Tsinghua University and the PhD degree in computer science from UCLA. He is a research scientist with Beijing Institute for General Artificial Intelligence (BIGAI). He is passionate about building models/agents that can learn from 2D/3D vision and text data, and perform a wide range of reasoning, embodied planning and control tasks.



Zilong Zheng (Member, IEEE) received the PhD degree from the University of California, Los Angeles (UCLA). His research focuses on generative modeling, cognitive dialogues, and large language models. Dr. Zheng has made several original contributions to multimodal perception and interaction. He has published over 30 papers in prestigious journals and conferences such as Science Robotics, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, CVPR, ICLR, and ACL. Notably, his research on "In situ Bidirectional Human-Robot Value Alignment" was featured as a headline publication on Science Robotics.

was featured as a headline publication on Science Robotics.



Yitao Liang (Member, IEEE) received the PhD degree in computer science from UCLA, advised by Prof. Guy Van den Broeck. He is an assistant professor and Boya Fellow with Peking University. His research interests span knowledge reasoning, machine learning and AI agents. His work has received recognition from top AI conferences; for example, the best-paper honorable mention from AAMAS 2016, the best paper from RL for Real Life workshop in ICML 2019, a best paper runner-up from the LLD workshop in NeurIPS 2017, a best paper from the TEACH workshop in ICML2023. He regularly serves as area chairs and senior area chairs in top venues e.g., NeurIPS, ICML.

TEACH workshop in ICML2023. He regularly serves as area chairs and senior area chairs in top venues e.g., NeurIPS, ICML.